
Dataset Condensation for Time Series Classification via Dual Domain Matching

Medha Bhavani Girish

EE23B112

DA5400, Foundations of Machine Learning

Abstract

Time series data plays an important role across various domains, but training deep neural networks on large scale temporal datasets comes at a large computational and storage cost. Dataset Condensation addresses this by synthesizing a small, highly informative dataset from this huge dataset that achieves a comparable performance to the original huge one. While methods for this exist already, the primary drawback of using conventional image-based data condensation methods for time-series data is that they ignore critical dynamical information in the frequency domain. The Dataset Condensation for Time Series Classification via Dual Domain Matching (CondTSC) method implemented in this report leverages matched gradients over the time and frequency domains to synthesize datasets at extreme compression rates. The results show that the proposed method, CondTSC, is able to achieve similar levels of accuracy compared to the full-dataset baselines (SVM, Random Forest, and KNNs) with less than 5% of the original dataset size. It can further transfer to architectures with shared convolutional inductive biases.

Project Repository: https://github.com/LucyStellar07/Dataset_Condensation/tree/main

Video Presentation: <https://youtu.be/u3y90LLTiE0>

1 Motivation

Time series data is present everywhere- in IoT sensors, health monitors, financial tracking systems, digital signal processing, and so many other applications. While deep learning architectures like CNNs, LSTMs and transformers are well established for their state-of-the-art performance in handling time series prediction and classification, their success is heavily reliant on massive quantities of data. Handling these huge datasets presents several challenges with respect to storage and latency.

Dataset Condensation has emerged as an effective solution to this problem. Instead of simply sampling a subset of the dataset, which often leads to loss of rare class boundaries, or compressing files and losing context, dataset condensation uses bi-level optimization to synthesize a small, artificial dataset. The fundamental idea here is that a neural network trained on this much smaller synthetic dataset should theoretically be able to achieve the exact same parameter distribution as the original large one, and hence achieve the same accuracy metrics.

While this idea is extremely effective, existing dataset condensation methods have been designed exclusively for spatial image data, and yield sup-optimal performances when applied to time-series data. Time series data stands out from other formats of data like images in that it possesses a fundamental duality- it contains structural information in both the time and frequency domain, and both should be exploited to gain more information about the data itself. Motivated by this difference, this report reproduces the "Dataset Condensation for Time Series Classification via Dual Domain Matching (CondTSC)" paper, aiming to achieve the same levels of performance in the paper in our

own experiments and simulations, proving that condensation of time-series datasets by exploiting their information in both the time and frequency domain is a powerful method to optimize storage and speed while training deep learning models.

On a personal note: I am a student in the Electrical Engineering department, and I have a keen interest in communications and signal processing. My undergraduate research so far has been primarily centered around optimizing the performance of analog-to-digital converters, and I have been working on using LSTMs and transformers to mitigate the distortions introduced in the signal in the digitization process. This paper on dataset condensation stood out to me because of its direct relevance in the research work I am doing- I am modeling the denoising of signals as a time series prediction problem, and this proposed method of condensation by using the dual domain nature of time-series data is something I found quite interesting. I mention this as a note to not only describe the motivation of this project in a technical sense, but also to also mention what motivated me personally to choose to implement this paper.

2 Problem Statement

Let the real time-series training dataset be defined as $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^{|\mathcal{T}|}$, where the input time-series sample is represented as $x_i \in \mathbb{R}^{C \times L}$. The objective of Dataset Condensation is to generate a compact synthetic dataset $\mathcal{S} = \{(s_i, c_i)\}_{i=1}^{|\mathcal{S}|}$ such that $|\mathcal{S}| \ll |\mathcal{T}|$.

The optimization objective is formulated as $\min_{\mathcal{S}} \mathbb{E}_{(x,y) \sim P_{\mathcal{D}}} [\ell(f_{\theta^{\mathcal{S}}}(x), y)]$ subject to $\theta^{\mathcal{S}} = \arg \min_{\theta} \mathcal{L}_{\mathcal{S}}(\theta)$.

The variables used in the above equations are defined as follows:

- (x_i, y_i) : The i^{th} real training sample and its corresponding class label.
- $x_i \in \mathbb{R}^{C \times L}$: Multivariate time-series sample with C channels and sequence length L .
- $y_i \in \{0, 1, \dots, K-1\}$: Ground truth class label corresponding to x_i .
- (s_i, c_i) : Synthetic sample and its associated synthetic label.
- $s_i \in \mathbb{R}^{C \times L}$: Synthetic time-series sample having same dimensionality as original data.
- f_{θ} : Neural network parameterized by weights θ .
- $\theta^{\mathcal{S}}$: Model parameters obtained after training on the synthetic dataset $\mathcal{S}$.
- $P_{\mathcal{D}}$: Distribution of the real data.
- $\ell(\cdot)$: Classification loss function.
- $\mathcal{L}_{\mathcal{S}}(\theta)$: Training loss computed on the synthetic dataset.
- IPC (Instances Per Class): Number of synthetic samples per class in the condensed dataset.

2.1 Gradient Matching in Dual-Domain

The paper proposes that instead of directly solving the minimization problem presented above, which is a very computationally intensive process, we instead use Gradient Matching as an equivalent objective. The authors of the paper hypothesize that if the gradients produced by the synthetic dataset are approximately equal to the gradients produced by the original massive dataset at every step of the training, the resulting network weights will also be approximately equal.

Another feature implemented is exploiting the dual-domain nature of the time-series data. The proposed method, CondTSC, projects the data in both time and frequency domains via an FFT. Let $f = h_{\phi} \circ g_{\psi}$ be our network, where g is the feature extractor and h is the classification head. If the time domain data is represented as F_T and frequency domain data as F_F . The dual-domain embedding is concatenated as $E = [F_T \oplus F_F]$.

At training step t , we sample a batch of real data $\mathcal{B}_k^{\mathcal{T}}$ and synthetic data $\mathcal{B}_k^{\mathcal{S}}$ for class k . The dual-domain gradient matching loss is defined as the distance between the gradients of the classification loss ℓ with respect to the network weights θ :

$$D(\mathcal{S}, \mathcal{T}) = \sum_{k=0}^{K-1} \sum_{t=0}^{T_{steps}} \|\nabla_{\theta} \ell(f_{\theta_t}(\mathcal{B}_k^{\mathcal{T}}), y_k) - \nabla_{\theta} \ell(f_{\theta_t}(\mathcal{B}_k^{\mathcal{S}}), c_k)\|_2^2 \quad (1)$$

With the objective of minimizing this loss, we iteratively update s_i to effectively encode the dual-domain properties of the original huge dataset $\mathcal{T}$. The paper also implements K-Center initialization rather than random noise to give a geometrically optimal starting point for the synthetic data before performing gradient matching.

3 Description of Datasets

The effectiveness and generalization capability of the proposed method was evaluated by implementing this method on 2 different datasets (both obtained from the UC Irvine Machine Learning Repository).

1. Human Activity Recognition Using Smartphones: This dataset contains inertial sensor data (linear acceleration and angular velocity along x, y, z axes) captured from smartphones.

- *Dimensions:* 9 channels, 128 time-steps per window.
- *Classes:* 6 classes (Walking, walking upstairs, walking downstairs, sitting, standing, lying down).
- *Suitability:* This dataset was chosen because human movement is a time-series type data that is fundamentally periodic- walking especially is a cyclic movement, which means it contains significant information in both the time and frequency domain.

2. Synthetic Control Chart Dataset: This dataset consists of synthetically generated control charts modeling various industrial process configurations, generated using the deterministic simulation process.

- *Dimensions:* 1 channel, 60 time-steps per sequence.
- *Classes:* 6 operational trends (Normal, Cyclic, Increasing Trend, Decreasing Trend, Upward Shift, and Downward Shift).
- *Suitability:* Unlike the localized multi-channel patterns in the human activities dataset, the Control Chart dataset is characterized by more global geometric configurations—such as linear trends and discrete step-function offsets, letting us evaluate how well the dual domain approach works in this kind of environment as well.

4 Experimental Set-up and Baselines

The proposed algorithm was implemented in PyTorch. A co-training bi-level optimization loop was employed for the generation of the synthetic dataset. The outer loop updates the synthetic dataset by minimizing the defined loss function over 500 epochs, while the inner loop updates the 'Teacher' network to ensure the synthetic dataset learning is following a realistic learning trajectory compared to that of the original huge dataset.

A note: Due to the high compute requirements for calculating second-order derivatives through deep recurrent layers, I was able to implement only gradient matching for the final layer- gradients were matched exclusively across the final MLP classification layer of the embedding space.

Baselines: I am comparing the performance obtained from the condensed dataset to that obtained from 3 traditional ML algorithms that are trained on the original huge dataset. So to clarify, we are going to be comparing the results we get from 3 standard algorithms implemented on 100% of the dataset to the results we get from our proposed algorithm implemented on 5% of the dataset. We use SVMs, KNNs and Random Forests as our 3 baseline algorithms to compare performances.

1. **SVMs:** Used with a Radial Basis Function kernel to evaluate maximal margin separation. In the context of time-series classification, the kernel projects temporal features into a high-dimensional space to resolve non-linear class boundaries. Since SVMs rely entirely on the support vectors, they tend to be extremely accurate in making decisions on classification.
2. **Random Forests:** An ensemble learning method constructed from several decision trees, that uses bagging to mitigate overfitting and improve generalization. Because Random Forests rely on statistical splitting criteria rather than gradient-descent optimization, they provide a good baseline to compare with.

3. **KNNs:** A distance-based classifier ($k = 5$ in our case) that provides a baseline for raw spatial clustering. KNN classifies samples based purely on the local density and Euclidean proximity of the training data in the feature space, giving us a performance baseline that relies entirely on just the geometric distance.

These baselines establish the target accuracy. If the deep network trained on synthetic data that is 5% of the original dataset size can approximate the accuracy of these algorithms trained on 100% of the original data, we can accept our method to be successful.

5 Metric for Comparison

The primary metric we are using to evaluate performance is the accuracy obtained by testing our model on an unseen testing dataset.

$$\text{Accuracy} = \frac{TP + TN}{P + N} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Test Samples}} \quad (2)$$

The original dataset is into 80-20 for training and testing. After training with the 80 percent, it is tested on the unseen 20 percent, ensuring that its performance is evaluated based on its generalization capability rather than memorization.

6 Results of Simulation Studies

6.1 Main Performance and Data Compression

Our primary experiment tested the algorithm on the human activities dataset. Using an Instances Per Class (IPC) of 50, we condensed the 7352 real training samples down to 300 synthetic samples (a compression ratio of $\sim 4\%$). When trained only on these 300 synthetic samples, our TCN achieved an accuracy of 82.54%. This performance outperformed the full-dataset KNN baseline (71.2%) and is comparable to the Random Forest (84.0%), and SVM (91.0%).

6.2 Cross-Architecture Generalization

For dataset condensation to be effective across different use cases, the synthetic data produced must be agnostic to different architectures- it should be able to train different neural networks regardless of the underlying structural paradigms. An architectural bias based on how the synthetic data is generated could degrade its value significantly if it only works well on the exact architecture that was used to generate it. To validate the universal applicability of our synthesized dataset, we compare the performance across different architectures, as shown in Fig. 1. In particular, we used a 1D-CNN model as the “Teacher” to generate the data representations and then rigorously tested the quality of that data by training four different architectures from scratch. The IPC used for this simulation is 30 (representing 180 samples, or 2.4 percent of the original dataset).

The synthetic data generalized well to the Temporal Convolutional Network (TCN) achieving 77.7%. However, performance degraded on MLPs and LSTMs.

These results successfully reproduced the core behavioral trends established by the authors in the paper, although the levels of optimization achieved are lower due to compute constraints.

- The authors claim that CondTSC achieves the best performance, especially in the CNN-based models. The results from our implementation achieve similar trends. When the synthetic dataset was generated using a CNNBN “Teacher” architecture, it showed good transfer capability to TCNs and CNNs, since these share the spatial sliding-window inductive biases of CNNs.
- The authors also noted that for structurally distinct architectures like RNNs and MLPs, standard single-step gradient matching methods are more unstable. My results show this same trend as well. When evaluating the CNN-generated synthetic data on an LSTM, the accuracy dropped down to 55.5 percent- without proper regularization techniques, the model overfits to the teacher architecture.

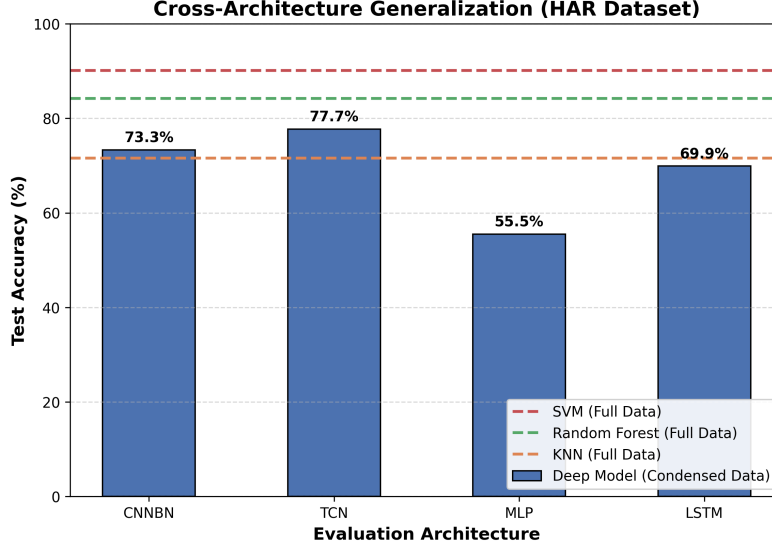


Figure 1: Cross-arch generalization on the human activities dataset (synthetic dataset is 2.4 percent of the original dataset size)

- These results shown above are achieved with a dataset of size 180, which is 2.4 percent the size of the original dataset. The paper uses only 1 percent of the samples to achieve similar results. This discrepancy is due to lack of compute power. My implementation of this paper uses a 1-to-1 optimization loop and applies gradient-matching only in the final MLP layer, as I kept running into out-of-memory failures otherwise. The original paper possessed the computational resources to run multi-step trajectory loops, matching gradients across the entire deep network graph. As a result, while our results prove the hypothesis is correct, a slightly larger synthetic dataset was required to compensate for the simplified version of implementation.

6.3 Analysis of Impact of IPC (Compression Ratio)

A fundamental trade-off in Dataset Condensation is balancing the extent of compression of the dataset against the retention of critical features. To evaluate this, we analyzed the compression ratio to the accuracy achieved. As illustrated in Figure 2, increasing the IPC provides a steady improvement in accuracy. At low IPCs, the synthetic data is able to capture the dominant frequency information, but not the subtle intra-class variances. Consequently, accuracy remains relatively low since the network cannot establish sharp inter-class decision boundaries. However, as the IPC increased to 30 and 50, the gradient matching utilizes these additional synthetic instances to give a better encoding.

6.4 Ablation Study: The significance of frequency domain information

One of the author’s primary claims is that the information stored in the frequency domain in time-series data is crucial for encoding it into a compressed representation. To validate this, I have performed an experiment by comparing our achieved performance to the performance obtained from purely the time domain information, by zeroing out the F_F embeddings during the gradient matching phase, as seen in Fig. 3. Neglecting the frequency matching reduced the achieved accuracy from 82.54% to 56.8%, showing the importance of the dual-domain approach.

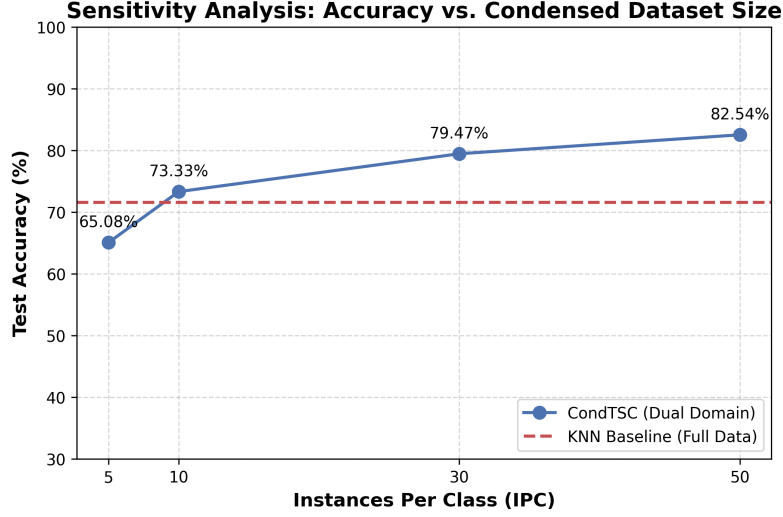


Figure 2: Accuracy as a function of IPC

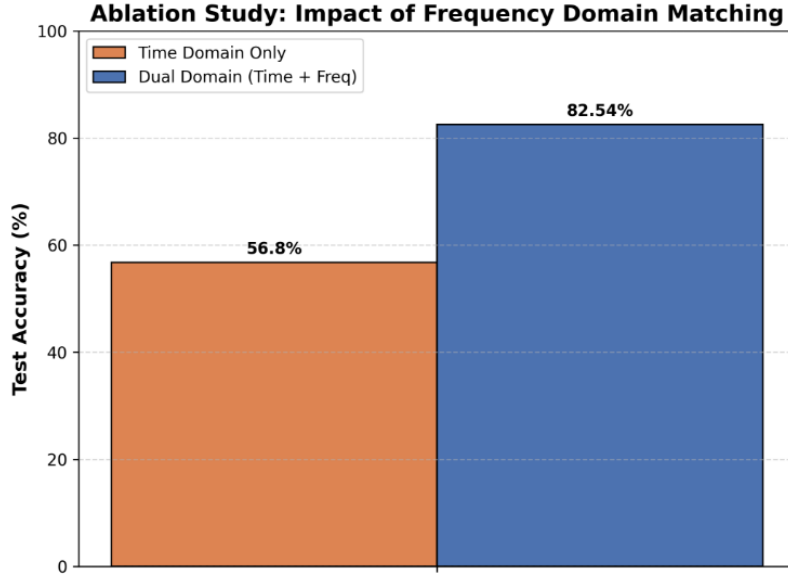


Figure 3: Decline in accuracy observed when only time domain information used

7 Performance and Evaluation on Synthetic Control Chart Dataset

We further validate our experiments on a second dataset as well- the Synthetic Control Chart time-series dataset containing 600 univariate sequences in 6 classes representing standard industrial manufacturing states- Normal, Cyclic, Increasing Trend, Decreasing Trend, Upward Shift, and Downward Shift.

The full-dataset baselines had a highly accurate performance- SVM achieving 97.4%, Random Forest reaching 97.0%, and KNN achieving 96.0%. Despite the extreme compression imposed by the condensation algorithm, the deep neural networks trained exclusively on the synthesized data yielded remarkable performance: the MLP achieved 94.9%, the CNNBN achieved 92.8%, and the TCN reached 92.3%, despite reducing the size of the dataset to 4 percent of its original size.

This is a rather interesting result. On the human activities dataset, the synthetic data’s performance strictly followed the teacher network training- CNNs and TCNs performed better given their inductive

bias, while LSTMs and MLPs performed worse. However, the synthetic control chart dataset reverses this trend- showing a similar, but better performance for the MLP.

Consequently, these results revise our previous analysis- while the architecture of the teacher network does play a major role in the performance of the synthetic data, the inherent geometry and dimensionality of the target dataset also play a role in deciding whether the condensed dataset behaves in a architecture-agnostic manner or not.

8 Discussion on Results

Thus, we have been able to implement and validate the author’s claims in the paper. Analysing these further:

8.1 Information Compression

The deep learning model trained on only a small fraction of the dataset (we have shown experiments for IPC varying from 5 to 50, which is between 0.4 and 4 percent of the original dataset size) achieved 82.54% accuracy on the human activities dataset. This is a highly efficient compression, giving comparable results to baseline algorithms at a significantly lower computation power.

These results demonstrate that this condensation via gradient matching is quite different from simply downsampling or heuristic based feature selection. The synthetic data generated is a highly informative, compact representation, specifically generated to maximize the mutual information between the original dataset’s distribution and its own- giving a close approximation of the full dataset while using minimal number of parameters.

8.2 Duality

The ablation study proves the necessity of using the information in the frequency domain. We see a significant decline in performance when the frequency information is not exploited. This behavior is mathematically justified by the structural characteristics of time-series information. Standard dataset condensation algorithms operate only within the temporal domain, and hence, while they optimize local sample alignment, they often suffer from phase erasure or high-frequency distortion. However, since we use the frequency content of the signal as well, our algorithm is able to maintain long-range periodic properties and spectral amplitudes.

8.3 Architecture Transfer Capability

We ideally want a condensation algorithm that is agnostic to the type of architecture used. We observe that while the performance is not entirely agnostic (degrading for LSTMs when trained using 1D-CNNs), the accuracy achieved is still an acceptable level.

This is somewhat expected, because an LSTM uses recursive matrices and hidden memory states, which is an entirely different approach compared to 1D-CNNs. This means that the synthetic data trained using the CNN was built to trigger maximize backpropagation gradients with spatial filters, but lacks the temporal smoothness that LSTMs make use of. Further optimization and regularization should be carried out.

9 Limitations

While I achieved similar result trends to that of the paper, I faced some drawbacks in implementation:

1. Compute and storage limitation: Trying to match gradients over the entire deep network resulted in CUDA Out-of-Memory errors on my system. I was forced to limit gradient matching to the last MLP classification layer to be able to run this- giving me suboptimal results compared to those presented in the paper.
2. Initialization based overfitting: The synthetic data is highly sensitive to overfitting the specific initialization of the teacher network. To work around this, I implemented a co-

training loop that reset the weights periodically to ensure robustness and generalization capability.

3. Not entirely agnostic to architectures: As observed in the results, gradient matching naturally biases the data towards the teacher architecture. To achieve true cross-architecture generalization (like CNN to LSTM), DSAs or ensemble-averaged teachers can potentially be implemented.

10 Acknowledgements

I would like to thank my Professor, Nirav Bhatt, for giving me the opportunity to do this project- it was a good learning experience. I explored a lot of different topics across the course of this project- even just selecting which paper to implement gave me a lot of exposure to different topics of research in ML, and as I was searching for different ways to fix the issues I faced during implementation, I ended up browsing different websites that led me to learning about many different topics. I used LLMs to format the overleaf document without changing the content- mostly for the mathematical equations in particular. I also used LLMs to give me the syntax for generating the plots, as I wasn't too familiar with the matplotlib library functions. I also used LLMs to format my slide deck in the presentation. Other than that, the code and report are my own. Once again, my gratitude towards the professor for a fruitful project and a wonderful course.

References

- [1] Zhanyu Liu, Ke Hao. *Dataset Condensation for Time Series Classification via Dual Domain Matching*. Shanghai Jiao Tong University, 2026.
- [2] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. *A Public Domain Dataset for Human Activity Recognition Using Smartphones*. In Proceedings of the 21st European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN), 2013.
- [3] Robert Alcock. *Synthetic Control Chart Time Series*. UCI Machine Learning Repository, 1999. <https://doi.org/10.24432/C59G75>.